SHRI SANT GAJANAN MAHARAJ COLLEGE OF ENGINEERING, SHEGAON



Department of Computer Science and Engineering

Compiler Design | TEC (Teaching Evaluation Component) Assignment

Academic Year 2026–2027 | Third Year, Fifth Semester

PROJECT TITLE

Syntax Analyzer and Parse Tree Generator Using Python

Submitted by

Kartik Bhaskar Hande — Roll No. 50

Armaan Ramprakash Gupta — Roll No. 40

Devesh Girish Talatule — Roll No. 44

Divyansh Vijay Mate — Roll No. 45

Project guide: Prof. C. V. Manakar

Submission date: 30 October 2026

GitHub: https://github.com/kkartik1729/Compiler_design_project-

Abstract

This report describes an educational web application that demonstrates the front end of a small expression language. A hand-written lexer recognizes numbers, identifiers, operators, parentheses, and assignment. A recursive-descent parser checks an explicitly defined grammar and builds an abstract syntax tree (AST). The browser displays the token stream, tree, diagnostics, and optional three-address code generated from that tree. The application accepts expressions such as a + b * c and rejects malformed input with a position-aware message. Automated verification ran 13 unittest cases successfully, including lexer, parser, AST/code-generation, and Flask route checks.

Report Guide

Section	Content
1–4	Introduction, problem, objectives, scope, and requirements
5–7	Compiler concepts, grammar, system design, and data flow
8–10	Algorithms, error handling, implementation, and interface
11–14	Application screenshots, output, tests, results, and discussion
15–18	Conclusion, future work, repository details, references, appendix

1. Introduction

A compiler translates source programs through a sequence of analysis and translation stages. The front end determines how characters group into tokens and whether those tokens follow the language grammar. These early stages make later compiler work possible: a parser needs a token stream, and later semantic or code-generation phases need a structured representation of the program.

This project implements those front-end ideas for arithmetic expressions in Python. The interface accepts source text and sends it to a Flask endpoint. The lexer returns tokens with source locations. A recursive-descent parser applies precedence rules and constructs an AST. The application also emits three-address code by traversing the AST.

2. Problem Statement

Students often see an expression as a string without seeing the intermediate steps that explain how a compiler interprets it. A plain calculator result does not show token boundaries, operator precedence, parenthesis grouping, or the reason invalid syntax fails. The project addresses this learning need with a small language whose lexical and grammatical rules are explicit and whose analysis results remain visible.

3. Objectives

- Recognize numeric literals, identifiers, assignment, arithmetic operators, and parentheses.
- Validate expressions with a recursive-descent parser implemented for this project.
- Construct and visualize an AST whose grouping reflects precedence and parentheses.
- Report the first lexical or syntactic error with a line and column.
- Generate three-address instructions from the parsed tree.
- Present tokens, tree, diagnostics, grammar, and examples in a responsive web page.

4. Scope and Limitations

The implemented language is deliberately small. It accepts decimal numeric tokens, identifiers that begin with a letter or underscore, parentheses, unary minus, the binary operators +, -, *, and /, and an optional assignment with one leading identifier. Whitespace separates tokens but does not otherwise affect the grammar.

4.1. Supported and Unsupported Features

Supported	Not implemented
Integers and decimal numbers	Exponent notation and numeric evaluation
Identifiers using letters, digits, underscore	Unicode identifier rules or reserved words
+ - * / and unary minus	Unary plus, power, comparisons, Boolean operators
Parenthesized expressions	Function calls, arrays, multiple statements
One leading assignment	Chained assignment or assignment expressions
Position-aware first-error diagnostics	Error recovery, types, symbol tables, semantic checks

The program parses but does not evaluate arithmetic values. It does not resolve identifiers, perform type checking, or detect division by zero. A syntax tree is an AST: it keeps operators and operands while omitting grammar-only nodes such as Expr and Term. It is not a concrete parse tree.

4.2. System Requirements

Component	Requirement
Operating system	Windows, macOS, or Linux with a supported Python installation
Runtime	Python 3.10 or newer is recommended
Web framework	Flask 3.x; listed in requirements.txt
Browser	Current desktop or mobile browser with JavaScript enabled
Hardware	Any computer able to run Python and a modern browser; no external compiler executable

5. Theory and Concepts Used

5.1. Lexical Analysis and Tokens

A lexer reads the input character by character and groups characters into tokens. A token records a category, lexeme, line, column, and character offset. For example, a + b * c produces IDENTIFIER(a), PLUS(+), IDENTIFIER(b), MULTIPLY(*), IDENTIFIER(c), followed internally by EOF. The browser omits EOF from the table because it is a parser sentinel rather than source text.

5.2. Context-Free Grammar

A context-free grammar describes valid token sequences with production rules. Its nonterminals name syntactic categories and its terminals correspond to tokens or literal punctuation. A parser checks whether a token stream can be derived from the start symbol.

5.3. Recursive-Descent Parsing

Recursive descent assigns methods to grammar levels. The expression method handles addition and subtraction, term handles multiplication and division, unary handles prefix minus, and primary handles literals, identifiers, and parentheses. Each method consumes the token forms allowed at its level and returns an AST node.

5.4. Precedence and Associativity

Multiplication and division have higher precedence because Expr delegates each operand to Term; Term consumes * and / before Expr can finish. Addition and subtraction are left-associative because Expr loops and replaces the accumulated left node with a new parent node. Term uses the same pattern for multiplication and division. Parentheses recursively parse a full Expr and return its inner node, preserving the requested grouping.

5.5. AST and Intermediate Representation

An AST preserves meaningful operators and operands without retaining all grammar nodes. For a + b * c, the root is +, its left child is a, and its right child is *. A simple three-address representation introduces temporary names for computed subexpressions. The output is educational intermediate code; it is not machine code and is not executed.

6. Grammar Specification

The grammar below is the grammar implemented by parser.py. NUMBER and IDENTIFIER are token categories. Literal symbols are written in quotes.

```
Program   → Statement EOF
Statement → IDENTIFIER '=' Expr | Expr
Expr      → Term (('+' | '-') Term)*
Term      → Unary (('*' | '/') Unary)*
Unary     → '-' Unary | Primary
Primary   → NUMBER | IDENTIFIER | '(' Expr ')'
```

6.1. Production Rule Notes

Production	Meaning in the parser
Program → Statement EOF	Accept exactly one statement and require the end of input.
Statement → IDENTIFIER '=' Expr | Expr	Treat a leading identifier followed by '=' as one assignment; otherwise parse an expression.
Expr → Term (('+' | '-') Term)*	Parse left-associative addition and subtraction.
Term → Unary (('*' | '/') Unary)*	Parse left-associative multiplication and division at higher precedence.
Unary → '-' Unary | Primary	Allow recursive prefix negation.
Primary → NUMBER | IDENTIFIER | '(' Expr ')'	Recognize operands and parenthesized expressions.

6.2. Grouping Examples

Input	AST grouping
a + b * c	+(a, *(b, c))
(a + b) * c	*(+(a, b), c)
a - b - c	-(-(a, b), c)
-a + 5	+(-(a), 5)

The first example shows multiplication nested beneath addition. The second moves addition below multiplication because the parentheses form a nested primary expression. The third demonstrates left associativity.

7. System Design and Block Diagram

The application follows a linear compile-front-end path. Source text enters the lexer, whose tokens feed the parser. Successful parsing returns an AST. The AST is serialized for the browser, rendered as a tree, and traversed to generate three-address code. Lexical and syntax failures return an error object and bypass later stages.

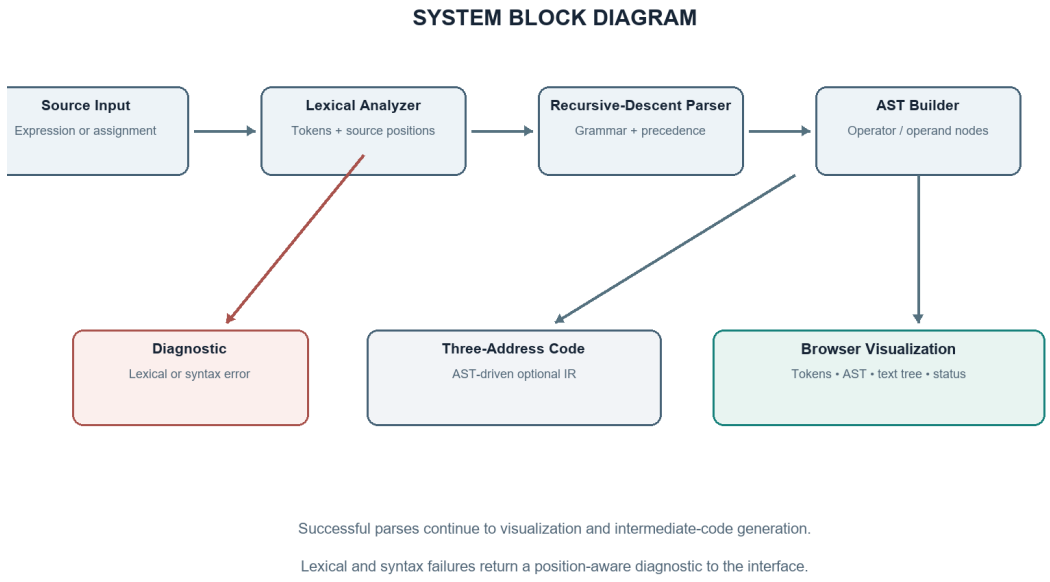


Figure 1. System block diagram for the implemented analyzer.

7.1. Main Modules

Module	Responsibility
lexer.py	Produces position-aware token objects and lexical diagnostics.
parser.py	Consumes tokens using recursive-descent methods and builds the AST.
ast_nodes.py	Defines immutable AST nodes, JSON serialization, and text-tree output.
code_generator.py	Walks the AST and returns three-address instruction lines.
app.py	Serves the web page and the JSON analysis endpoint.
templates / static	Provides HTML, CSS, and browser-side rendering of API data.

7.2. Data Flow

POST /api/analyze accepts source text. Lexing runs first, followed by parsing. Success returns tokens, a nested AST, text-tree output, and code lines. Errors include a stage label and source position. Syntax failures retain the lexer's tokens for display.

8. Algorithms and Working

8.1. Tokenization Algorithm

- Initialize the source offset and one-based line and column counters.
- Skip whitespace, advancing the line counter after a newline.
- When a digit or decimal-point followed by a digit is found, scan one numeric lexeme.
- When a letter or underscore is found, scan the complete identifier.
- Map supported single-character operators and delimiters to token types.
- If a character has no rule, return a lexical diagnostic at its current line and column.
- Append EOF after the input is consumed.

8.2. Recursive-Descent Algorithm

- Read one statement. If an identifier is immediately followed by assignment, consume both and parse the right-hand expression.
- Parse Expr by first parsing Term, then repeatedly consuming plus or minus and another Term.
- Parse Term by first parsing Unary, then repeatedly consuming multiplication or division and another Unary.
- Parse Unary recursively for a leading minus; otherwise delegate to Primary.
- Parse Primary as a number, identifier, or a parenthesized Expr followed by a required closing parenthesis.
- After the root node is built, require EOF so trailing tokens are rejected.

8.3. Tree Construction

Each parser method returns one AST node. A binary operator node stores its left and right subtrees. A unary node stores one child. An assignment node stores the target identifier and parsed expression. Literal and identifier nodes have no children. The browser recursively creates DOM elements from the returned tree data, using textContent so token lexemes are not treated as HTML.

9. Flowchart and Error Handling

The flowchart shows the control path from the source string to the displayed output. The lexer and parser stop at the first error; the endpoint returns a structured diagnostic instead of allowing an exception to crash the request.

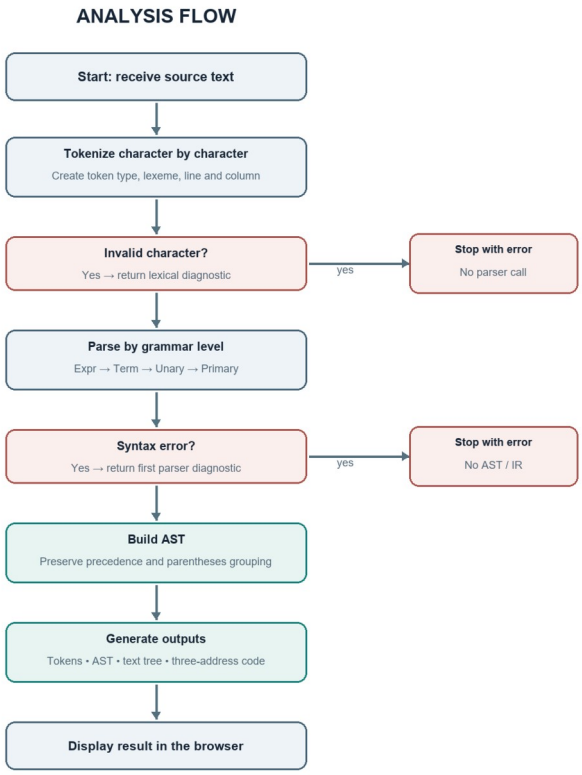


Figure 2. Analysis flow and early error exits.

9.1. Implemented Diagnostics

Condition	Observed response behavior
Empty source	Syntax-stage message asks the user to enter an expression.
Unsupported character	Lexical-stage message identifies the character and position.
Missing operand	Parser reports the token position where an operand was required.
Missing closing parenthesis	Parser reports the current token position and expected ')'.
Adjacent operands	Parser reports an unexpected token and suggests an operator may be missing.
Incorrect assignment	Only a leading identifier followed by one '=' is accepted.
Division by zero	Not checked because this project does not evaluate values.

Diagnostics identify the first failure only; the parser does not attempt recovery or report multiple errors in one request.

10. Implementation Details

10.1. Position-Aware Tokens

Token is an immutable data class. The lexer stores both user-facing line and column values and a zero-based character offset for programmatic use.

```
@dataclass(frozen=True)
class Token:
    type: str
    lexeme: str
    line: int
    column: int
    offset: int
```

10.2. Precedence Parsing

The two loop-based methods encode the operator precedence ladder directly. The expression layer only consumes additive operators; the term layer consumes multiplicative operators.

```
def expression(self):
    node = self.term()
    while (op := self.match("PLUS", "MINUS")) is not None:
        rhs = self.term()
        node = ASTNode("BinaryExpression", op.lexeme, (node, rhs), op.line, op.column)
    return node

def term(self):
    node = self.unary()
    while (op := self.match("MULTIPLY", "DIVIDE")) is not None:
        rhs = self.unary()
        node = ASTNode("BinaryExpression", op.lexeme, (node, rhs), op.line, op.column)
    return node
```

10.3. AST-Driven Intermediate Code

The code generator recursively visits children before emitting the parent operation. This postorder traversal ensures that a nested multiplication is assigned a temporary before the addition that uses it.

```
if current.kind == "BinaryExpression":
    left, right = visit(current.children[0]), visit(current.children[1])
    counter[0] += 1
    temp = f"t{counter[0]}"
    lines.append(f"{temp} = {left} {current.value} {right}")
    return temp
```

10.4. Source File Map

Path	Purpose
app.py	Flask routes and integration of lexer, parser, generator.
lexer.py / parser.py	Lexical and syntax analysis.
ast_nodes.py / code_generator.py	Tree structure, serialization, intermediate code.
templates/index.html	Accessible page structure and visible grammar.
static/css/style.css	Responsive visual layout and component styles.
static/js/app.js; tests/	Browser rendering plus lexer, parser, and endpoint tests.

11. User Interface Design and Successful Output

The page keeps the source editor, status, tokens, AST, intermediate code, and grammar in one view. Example buttons demonstrate grouping, assignment, unary minus, and error handling. The successful screenshot below was captured from the running Flask application after analyzing a + b * c.

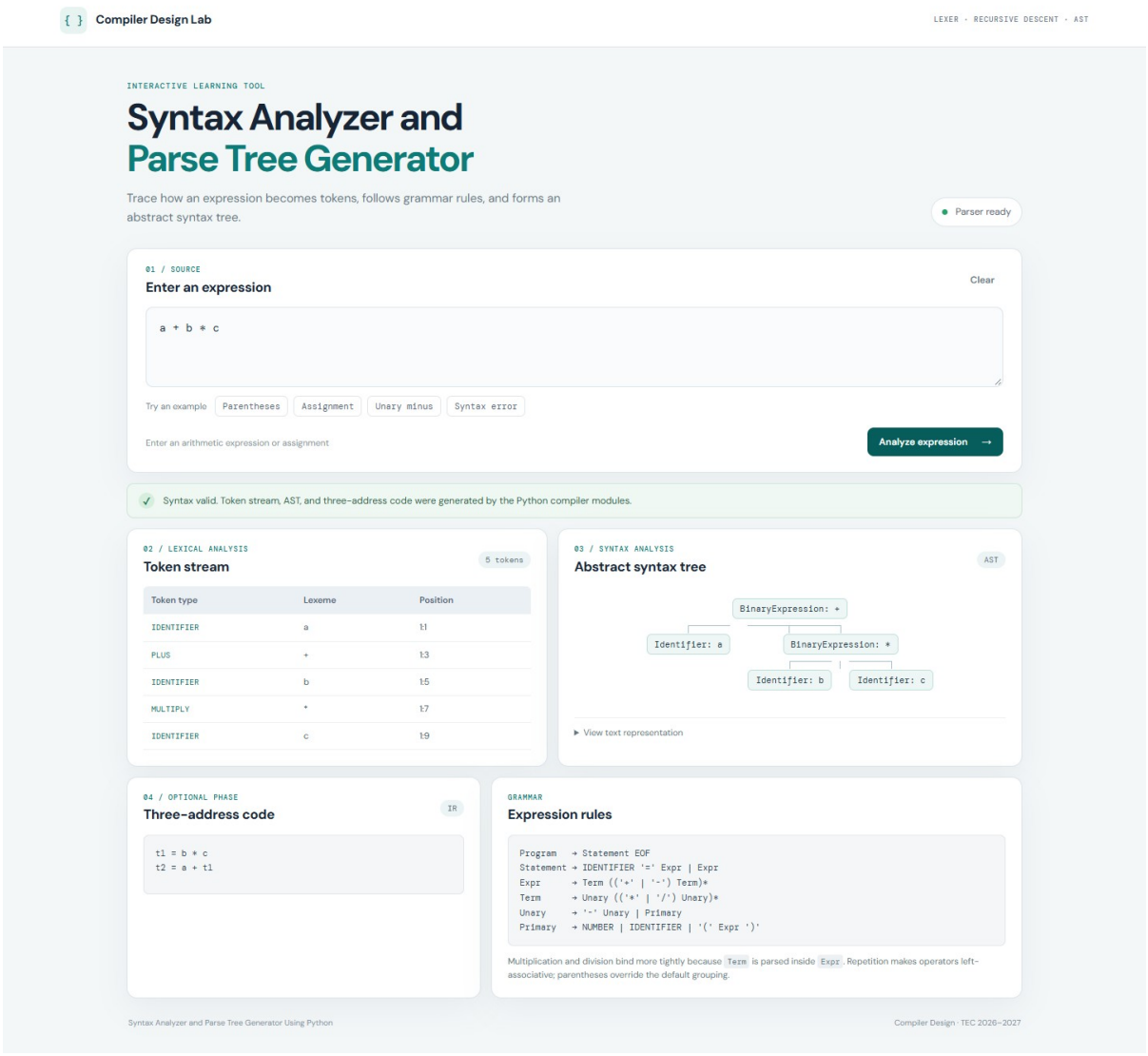


*Figure 3. Successful analysis of a + b * c in the running application.*

The result lists five source tokens with line and column, draws + above the nested multiplication node, and emits t1 = b * c followed by t2 = a + t1. This output demonstrates the precedence encoded in the grammar.

12. Invalid Input Output

The invalid screenshot shows a + * b. The lexer tokenizes the entire expression, then the parser reports that an operand was expected at line 1, column 5. No AST or intermediate code is produced for the invalid statement.

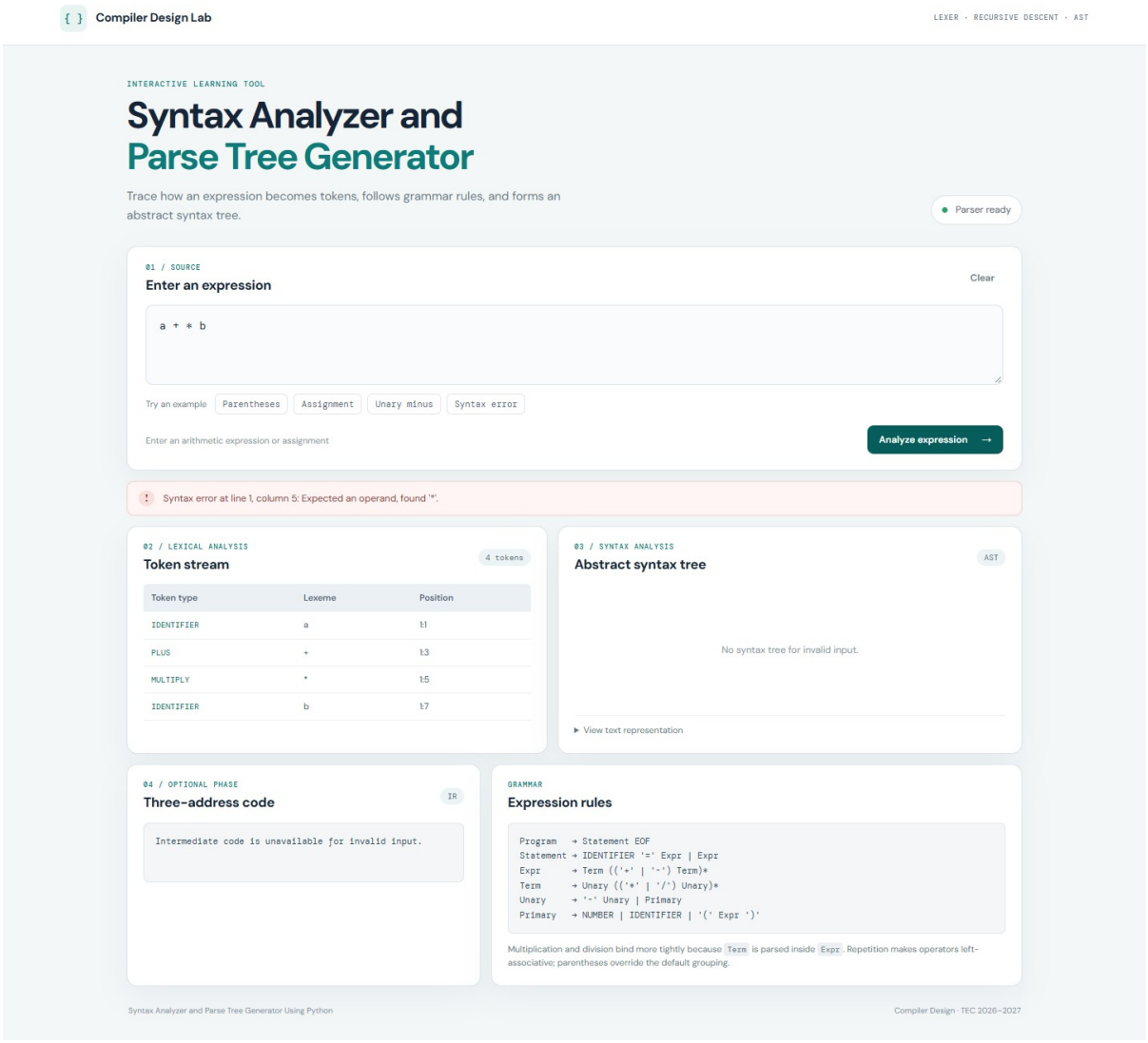


*Figure 4. Syntax error and retained token stream for a + * b.*

12.1. Interface States

State	Visible behavior
Initial	Prompt, example expression, empty result areas, ready status.
Successful	Green status, token table, AST, text tree, generated code.
Lexical failure	Red status with character position; later phases are skipped.
Syntax failure	Red status with parser position; tokens remain visible; tree and code are withheld.
Clear	Resets the input, outputs, and status to the initial state.

13. Output and Testing

The test suite uses Python's standard unittest framework. The result below records the latest run from the project root using the project requirements. All 13 tests passed.

Input / request	Expected behavior	Actual result	Purpose
a + b * c	Valid; * nested below +	Passed; AST and t1/t2 code returned	Precedence
(a + b) * c	Valid; + nested below *	Passed	Parentheses
10 / 2	Valid division expression	Passed	Division
-a + 5	Valid unary minus	Passed	Unary parsing
x = 10 + 20	Valid assignment	Passed	Assignment
a + * b	Reject at *	Passed; line 1, column 5	Unexpected operator
(a + b	Reject missing)	Passed; parser diagnostic	Delimiter error
10 /	Reject missing operand	Passed; EOF diagnostic	Incomplete expression
@a + 2	Reject at @	Passed; lexical position 1:1	Invalid character
empty input	Validation message	Passed; HTTP 400 syntax response	Empty source
a b	Reject adjacent operands	Passed; unexpected token	Missing operator
()	Reject empty parentheses	Passed; expected expression	Empty group

The table distinguishes expected behavior from the actual suite result. Flask tests exercised the home page, a successful JSON analysis, an invalid syntax response, empty input, and non-text input.

14. Results and Discussion

14.1. Test Run Summary

Test area	Checks	Result
Lexer	token categories, source position, decimals, invalid symbol	Passed
Parser and AST	precedence, grouping, unary minus, assignment, malformed inputs	Passed
Intermediate code	postorder temporary generation from AST	Passed
Flask integration	HTML route, successful API, error API, empty and non-text values	Passed
Overall	13 unittest cases	13 passed, 0 failed

14.2. Discussion

The implementation demonstrates how a grammar's structure controls the parser's call order. It makes precedence visible in both the AST and the generated code. Position-aware token and parse errors make malformed input easier to discuss than a generic failure message. The browser interface presents compiler stages together and preserves token output when syntax analysis fails.

The current implementation deliberately stops at syntax analysis and a small educational intermediate representation. It does not evaluate variables, check types, recover after an error, or detect division by zero. These are scope limits rather than successful claims about implemented semantic analysis.

15. Conclusion

The project delivers a working educational syntax analyzer built with Python and Flask. Its lexer recognizes the supported token categories and records their positions. Its recursive-descent parser implements the published grammar, constructs an AST with correct precedence and grouping, and reports a first error when the syntax is invalid. The interface displays actual server-generated results, and the code generator emits three-address instructions from the AST.

The test run passed 13 cases across the lexer, parser, AST/code generation, and Flask endpoints. These results support the stated behavior for the implemented expression language. The project provides a concrete basis for explaining tokenization, grammar productions, recursive-descent methods, operator precedence, ASTs, and an additional intermediate-code phase.

16. Future Scope

- Add unary plus, exponentiation, comparisons, Boolean operators, and multiple statements through explicit grammar extensions.
- Add symbol tables, type checks, and optional constant or division-by-zero checks as a separate semantic phase.
- Improve diagnostics with recovery strategies and multiple errors per input.
- Add syntax highlighting, keyboard shortcuts, downloadable trees, or alternative tree layouts.
- Extend intermediate code with labels, branches, and a small virtual machine while keeping execution isolated from Python eval.

17. GitHub Repository and Project Information

Repository URL: https://github.com/kkartik1729/Compiler_design_project-

The repository contains the Flask application, compiler modules, frontend assets, automated tests, README, screenshots, diagrams, and this report in PDF and editable DOCX forms.

17.1. Installation and Run Instructions

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python app.py
```

Open http://127.0.0.1:5000 in a browser. To run the tests, use:

```
python -m unittest discover -s tests -v
```

17.2. API Example

```
POST /api/analyze
Content-Type: application/json

{"source": "a + b * c"}
```

A successful response contains token objects, AST node objects, text-tree output, and three-address lines. An invalid response uses HTTP 400 and identifies whether the failure occurred during lexical or syntax analysis.

18. References

1. Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/
2. Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/
3. Python Software Foundation. unittest — Unit testing framework. https://docs.python.org/3/library/unittest.html
4. Python Software Foundation. dataclasses — Data Classes. https://docs.python.org/3/library/dataclasses.html
5. Shri Sant Gajanan Maharaj College of Engineering. TEC report supplied as a visual formatting reference, Tec_Report1.pdf. The reference was used for layout only; its project text and results were not reused.

Appendix A. Setup and Manual Verification

1. Install the dependency listed in requirements.txt.
2. Start the app with python app.py.
3. Open the local URL and analyze a + b * c.
4. Confirm the token table shows IDENTIFIER, PLUS, IDENTIFIER, MULTIPLY, IDENTIFIER.
5. Confirm the AST root is + and the right child is *.
6. Enter a + * b and confirm the parser reports the location of *.
7. Run the unittest discovery command and compare the output to the latest verified result.

Appendix B. Known Limitations

The parser reports a single error and does not recover. The language does not include type checking, variable lookup, expression evaluation, division-by-zero checks, exponent notation, function calls, comments, or multiple statements. The displayed tree is an AST. Temporary values are numbered in traversal order and are illustrative intermediate code.